NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

CAMPUS LEARNING MANAGEMENT SYSTEM

Software Engineering Project Report

CS4433 – Software Engineering Individual Assignment

Offline C++17 Console Application | SQLite | Windows Release 1.0.0

STUDENT

Libenyane Posholi

Student Number: 202321172

Abstract

This report documents the engineering and implementation of Campus Learning Management System (Campus LMS), an offline, menu-driven application developed in C++17 for the CS4433 Software Engineering individual assignment at the National University of Lesotho. The system is intended for use on a single school computer and supports four account roles: Administrator, Lecturer, Head of Department (HoD) and Student. Its core workflow covers local authentication, administration of departments and courses, lecturer assignment, student enrolment, assignment creation, submission and resubmission, individual deadline extensions, marking, moderation, release of results, audit-event recording and local backup. Persistent records are held in SQLite, while submitted files are copied into a local uploads directory. The Windows build was configured with CMake and compiled with Microsoft Visual C++; the Inno Setup script was compiled into a Windows installer and the installed application was launched and used for administrator login and account creation. The installed database was compared with the development database using SHA-256 and the values matched at the time of verification. This report also evaluates the solution against the assignment's broader software-engineering requirements. It distinguishes observed implementation and deployment evidence from work not demonstrated in the available source snapshot. In particular, no automated GoogleTest or Catch2 suite, CI workflow, line-coverage report or static-analysis report was supplied; the current source baseline concentrates the application workflow in one translation unit and represents the user as a role-bearing structure rather than an abstract polymorphic base class; the audit table records actor, action, reason and time but does not provide dedicated old-mark/new-mark fields; and the whole-class extension change request is not implemented in the observed release. These limitations are recorded rather than concealed. The report therefore serves as a truthful technical account, a traceability baseline and a prioritised completion plan for remaining assessment evidence.

Table of contents

- 1. Introduction and project overview
- 2. Project planning and management
- 3. Requirements engineering
- 4. Software architecture and detailed design
- 5. Implementation, database and security
- 6. Verification and validation
- 7. Configuration management and quality assurance
- 8. Deployment and operational documentation
- 9. Software maintenance and change request CR1
- 10. Professional, legal and ethical considerations
- 11. Critical evaluation and conclusion
- References
- Appendix A. Requirements traceability matrix
- Appendix B. Verification and test record
- Appendix C. Build and installation guide
- Appendix D. Outstanding compliance actions

Note: page numbers can be refreshed in Microsoft Word using References > Update Table after opening the document. The contents list is a navigation aid and the final pagination may change when edited.

REPORTING NOTE — EVIDENCE AND LIMITATIONS

This report separates (a) implementation details visible in the source baseline and project README, (b) build and manual outcomes directly observed during development and deployment, and (c) requirements or quality activities that remain unverified. Planning and requirements artefacts are reconstructed from the assignment brief where original signed documents were not supplied. The report must not be read as claiming that automated tests, coverage, CI, a v1.1 maintenance release or an abstract polymorphic user hierarchy have already been completed.

1. Introduction and project overview

Schools manage assessment information that must remain accurate, confidential and available to the people authorised to use it. A learning management system can reduce fragmented manual handling by bringing course administration, assignment deadlines, submission records and assessment decisions into a single controlled application. The case study for this project describes a secondary school with four user categories and an important separation of duties: lecturers mark submitted work, Heads of Department review and moderate those marks, and students see results only after release. The school also requires offline operation because internet access is unreliable and computers may be old.

Campus LMS was developed as a Windows console application rather than a browser-based or mobile service. The choice follows the assignment's explicit allowance for a console or simple desktop application on a single school computer or local network, while avoiding the operational dependency of a hosted web service. A user starts the program locally, chooses Login or student registration, and then receives a menu appropriate to the authenticated role. Administrative functions maintain users, departments, courses, lecturer assignments and enrolments. Teaching and assessment functions support assignment creation, individual extensions, submission, marking, moderation and publication of marks. Data is saved to a local SQLite database, and submitted documents are copied to a local folder managed by the application.

The project objective is broader than producing a menu that appears to work. It is to deliver a maintainable and explainable system whose requirements, design choices, implementation, verification and release can be reviewed. The assignment assesses planning, requirements engineering, detailed design, C++ implementation, verification and validation, configuration management, deployment, maintenance, and professional issues. The project is evaluated against these lifecycle expectations, not only against the visible application functions.

The implementation baseline examined for this report identifies C++17, CMake and SQLite as its core technologies. The CMake configuration requires C++17, disables compiler extensions, and either builds the bundled SQLite amalgamation or locates an installed SQLite package. On Windows, password key derivation and random salt generation use the Windows Cryptography API: Next Generation (CNG) through BCrypt. The application is organised around a central LmsApplication class, a Database class, a Statement wrapper, a PasswordHasher helper, and record structures for users, assignments and submissions. The source baseline contains src/main.cpp as the application translation unit. This is a compact implementation, but it also creates maintainability and design-compliance issues discussed later in this report.

Evidence basis and reporting boundary. This report uses the assignment brief, project README, inspected source/build snapshot and the Windows build and installation outputs shared during development. Planning and requirements artefacts not supplied as original documents are reconstructed and are identified as such. Automated tests, coverage, CI, a v1.1 release and an abstract polymorphic user hierarchy are not claimed. The later-added role headers were not available in the inspected source archive, so their integration remains to be checked.

2. Project planning and management

The assignment prescribes a seven-week Waterfall model supported by V-Model verification and validation. The expected phases are planning, requirements, design, implementation, testing, configuration/quality, deployment and maintenance. Phase gates reduce uncontrolled movement between stages, but they also make early requirements and design errors costly. The project therefore needs traceability from requirement to use case, design, source and test.

A retrospective work breakdown consists of scope and feasibility; stakeholder analysis and SRS; architecture, UML and database design; C++/SQLite implementation; unit, integration, system and acceptance testing; Git/CI/static analysis; installer and manuals; and CR1 maintenance. This sequence follows the assignment deliverables. It is a reconstructed plan because no dated original WBS was supplied.

A reasonable schedule places planning in week 1, requirements in weeks 1–2, design in weeks 2–3, implementation in weeks 4–5, verification in weeks 5–6, deployment in week 6, and maintenance/report completion in week 7. The critical path runs through requirements approval, design approval, the submission/moderation workflow, verification and packaging. Delays in any of these activities affect the final release.

This schedule is not proof that phase gates were formally approved. The assignment specifies seven weeks, but no dated Gantt chart, signed WBS or supervisor gate record was supplied. The report therefore reconstructs dependencies without inventing approval dates. Any genuine supervisor feedback and original schedule should be attached if available.

Technical feasibility is favourable for the development workstation. The project uses C++17, CMake and bundled SQLite; the Windows environment provided MSVC and a Windows SDK. The Release build succeeded on that computer. A clean-machine portability test has not been supplied, so compatibility beyond the development workstation remains to be verified.

Operational feasibility is reasonable for a single-computer school workflow. Role menus cover administration, teaching, moderation and student submission/results. Offline operation avoids reliance on internet access, but the school needs user training, controlled file permissions and reliable backups. The application is not a multi-computer server and should be operated within the stated single-computer scope.

Economic feasibility is qualitatively positive because the project uses local tools and does not require a hosted database or cloud subscription. However, no hardware quotation, development-time budget or lifecycle-cost worksheet was supplied. Costs are therefore limited to existing equipment, developer time, storage media and future support; no monetary saving is claimed.

Legal and ethical feasibility requires school controls over learner data. Lesotho’s Data Protection Act, 2011 (Act No. 5 of 2012) addresses processing, minimality, purpose, retention and information security [7]. The report is not a compliance certification. The school must define its data-controller role, lawful purposes, retention schedule, access permissions, backup protection and incident process.

Schedule feasibility is uncertain because only the seven-week assignment window is known; the student's actual dated schedule was not provided. A single developer can keep the scope manageable by prioritising the assessment lifecycle, then allocating time for testing and deployment evidence. New features such as CR1 should be managed as changes, not assumed to be part of release 1.0.0.

Risk management should be reviewed at each phase gate. The main risks concern data loss, unauthorised access, incorrect penalties, invalid file paths, weak backup/restore, workstation-specific builds, incomplete audit history and missing test evidence. Table 1 provides a reconstructed register with likelihood, impact, exposure, mitigation and owner; it is not a historical record of weekly risk updates.

2.1 Reconstructed work breakdown and schedule logic

Week	Main work packages	Dependency / milestone
1	Scope, feasibility, stakeholders, initial risk register	Project scope and problem statement baselined
1–2	Stakeholder analysis, conflict resolution, SRS, use cases	Gate 1: requirements review
2–3	Architecture, UML, class and behavioural models, ERD/schema	Gate 2: design review before full implementation
4–5	C++17 implementation, SQLite integration, role workflows	Core release candidate and build
5–6	Unit, integration, system and acceptance tests; defect correction	Gate 3: test evidence and acceptance review
6	Installer, manuals, deployment diagram and v1.0 release preparation	Deployment package prepared
7	CR1 impact analysis, implementation, regression and final viva/report	Maintenance baseline and project completion

2.2 Feasibility assessment

Dimension	Assessment	Main condition / evidence
Technical	Feasible for a local Windows desktop.	C++17, CMake, SQLite and Windows CNG; successful build on the development PC; clean-PC portability remains unverified.
Operational	Feasible for single-computer use.	Four role workflows and local data storage; requires training, file permissions and disciplined backup.
Economic	Qualitatively feasible.	Uses local/offline tools without cloud subscriptions; no itemised budget or total-cost analysis supplied.
Legal / ethical	Feasible only with institutional controls.	Learner data needs lawful purpose, minimal collection, access controls, retention and security measures.
Schedule	Possible but evidence incomplete.	Seven-week phase schedule from the assignment; original dated Gantt and gate approvals not supplied.

2.3 Risk register

ID	Risk	Likelihood	Impact	Exposure	Mitigation	Owner
R1	Loss or corruption of the local database or uploaded files	Medium	High	High	Administrator backup of the .db file and uploads together; test restoration; keep a copy on a separate protected drive.	Student developer / school administrator
R2	Unauthorised access to learner records or marks	Medium	High	High	Role checks, least privilege, salted password hashes, Windows file permissions, account review and security tests.	Student developer / school IT
R3	Incorrect late-penalty or deadline interpretation	Medium	High	High	Confirm policy with lecturer/HoD; test deadline boundaries; display effective deadline and penalty calculation.	Student developer / academic stakeholder
R4	Submission file path invalid or file copy fails	Medium	Medium	Medium	Validate regular files, show clear errors, test spaces/quotes/permissions, and preserve original file.	Student developer
R5	Insufficient automated test evidence and CI	High	High	High	Add GoogleTest/Catch2 target, CI workflow, coverage run and evidence archive before final submission.	Student developer
R6	Build succeeds only on development PC	Medium	High	High	Record compiler/SDK versions; perform clean-machine installation test; package all runtime dependencies.	Student developer / school IT
R7	Incomplete audit	High	High	High	Add structured old/new mark columns	Student

ID	Risk	Likelihood	Impact	Exposure	Mitigation	Owner
	history for mark changes				or immutable mark-history table and test every marking/moderation change.	developer
R8	Scope expansion delays the release	Medium	Medium	Medium	Prioritise core role workflow; manage CR1 separately; maintain phase gate and test time.	Student developer

3. Requirements engineering

The requirements baseline comes from the case study and stakeholder statements [1]. The Principal prioritises simple operation and HoD approval before publication; the Lecturer needs marking and extension capability; the HoD requires departmental moderation and late penalties; students need submission correction and feedback; and the IT Officer requires offline operation, security and acceptable performance on older equipment.

The Principal and Lecturer disagree about student access to marks. The implemented policy follows the Principal: lecturer marking changes the status to Marked, HoD review changes it to Moderated, and explicit HoD release changes it to Released. The student-results query returns only Released rows. The Lecturer can complete marking promptly, but publication remains controlled by the HoD.

Deadline extensions and resubmission are treated separately. A Lecturer can extend one enrolled Student's deadline by entering a later date and a reason. A Student can replace a submission only while its status is Submitted or Late and before it is marked. The current database stores one row per assignment/student pair, so resubmission replaces the active file reference rather than preserving all historical versions.

The HoD requests a 10% deduction of the assignment maximum per late day for up to three days, with zero after that. Students ask that late work be accepted. Campus LMS accepts late files, labels them Late and applies the deduction when marking. The code counts any positive delay as a started 24-hour period; this interpretation should be approved and communicated by the school.

The IT Officer's offline requirement supports a local executable and SQLite file. The system does not need a network service for normal use. The 800-student statement is a capacity expectation, not a measured result: the schema has indexes for common lookups, but no 800-user benchmark was provided. Performance at that scale remains a validation objective.

Table 2 presents a reconstructed functional requirement set derived from the case and observed implementation. MoSCoW priorities identify the minimum release workflow. These statements should be reconciled with any separately approved SRS before final submission. Out-of-scope items remain online payment, video conferencing, automated grading, plagiarism detection and web/mobile interfaces.

Non-functional requirements need measurable criteria. Offline operation can be checked by disconnecting the network; password handling can be checked by inspecting the stored salt/hash fields; and performance requires a defined machine and dataset. The assignment's 800-student context suggests a benchmark target, but the report does not invent a response-time result. Proposed targets are listed in Table 3 for future execution.

Use cases connect stakeholder expectations with acceptance tests. The report specifies submission, lecturer marking, and HoD moderation/release because they form the core assessment lifecycle. Their preconditions, normal flows, alternate paths, exceptions and postconditions can be used to create test scenarios. The remaining role interactions are summarised in Figure 1 and the traceability matrix.

The implementation validates menu ranges, integer and finite decimal inputs, date formats and impossible calendar values. Assignment maximum marks must be positive; awarded marks are bounded by zero and the maximum. Submission verifies assignment availability and checks that the source path identifies an existing regular file. These controls reduce malformed inputs but still need systematic negative testing.

The requirements model should be updated with each change. A change to roles, deadlines, penalties, submission history or the database must update its requirement, use case, design, schema, source and tests. CR1, whole-class deadline extensions, is not present in the inspected 1.0.0 source and is recorded as open rather than completed.

3.1 Functional requirements and priority

ID	Requirement statement	Priority
FR-01	The system shall authenticate a user using username and password and load the menu for the stored role.	Must
FR-02	The system shall allow an unauthenticated visitor to register a Student account.	Must
FR-03	The Administrator shall view users and create Lecturer, HoD and Student accounts.	Must
FR-04	The Administrator shall create and view departments and courses.	Must
FR-05	The Administrator shall assign lecturers to courses and assign HoDs to departments.	Must
FR-06	The Administrator shall enrol Students in courses and view course enrolments.	Must
FR-07	A Lecturer shall view only courses assigned to that lecturer account.	Must
FR-08	A Lecturer shall create an assignment with title, description, deadline and maximum mark for an assigned course.	Must
FR-09	A Lecturer shall grant an individual enrolled Student an extension with a later deadline and reason.	Must
FR-10	A Student shall view enrolled courses and assignments, including the effective individual deadline.	Must
FR-11	A Student shall submit a local regular file for an available assignment; the application shall copy it into local storage.	Must
FR-12	A Student shall resubmit an unmarked Submitted or Late item after confirmation; the stored submission row is replaced.	Should
FR-13	The system shall classify a submission as Submitted or Late using the effective deadline.	Must
FR-14	A Lecturer shall mark an eligible submission, provide feedback and apply the configured late penalty.	Must
FR-15	An HoD shall review marked work within the HoD department and enter a moderated mark and reason.	Must
FR-16	An HoD shall explicitly release a moderated mark; Students shall view only Released results.	Must
FR-17	The system shall record audit events for significant user and assessment operations.	Should
FR-18	A signed-in user shall change the password after verifying the current password; new passwords shall be at least eight characters.	Should
FR-19	The Administrator shall create a local database backup and a matching uploads backup.	Should

3.2 Non-functional requirements and acceptance criteria

ID	Measurable quality requirement / acceptance criterion	Evidence status
NFR-01 Offline operation	Normal login, course, assignment, submission, marking, moderation and release workflows shall run with the network disconnected.	Not yet measured as a separate offline acceptance test.
NFR-02 Password protection	No plain-text password shall be stored; each password shall use a distinct random salt and a PBKDF2-derived hash.	Implementation inspected; penetration/security review not performed.
NFR-03 Data integrity	Foreign-key and uniqueness constraints shall reject invalid relationships and duplicate student-course or assignment-student pairs.	Schema contains these constraints; negative DB tests not supplied.
NFR-04 Input validation	Invalid menu, numeric, mark and date input shall be rejected and re-prompted or reported without crashing.	Input validation present; exhaustive tests not supplied.
NFR-05 Performance	With 800 user accounts and representative enrolments, login	No 800-user benchmark supplied; target

ID	Measurable quality requirement / acceptance criterion	Evidence status
	and common list operations should complete within a school-agreed response target (proposed target: 2 seconds on the nominated desktop).	is proposed for validation.
NFR-06 Portability	A clean supported Windows desktop with the stated C++ runtime prerequisites shall install and launch the packaged release without CMake or source files.	Same-machine install tested; clean-machine portability not verified.
NFR-07 Recoverability	A backup set shall contain a consistent database copy and the associated uploads, and a restore exercise shall recover both.	Backup operation exists; restore drill not supplied.
NFR-08 Maintainability	The project shall build through documented CMake commands and keep generated data and build artefacts outside source control.	CMake Release build verified; architecture remains monolithic.
NFR-09 Confidentiality	Role access shall restrict student, lecturer and HoD views to their authorised records.	Role-scoped queries present; full negative authorisation suite not supplied.

3.3 Use-case overview

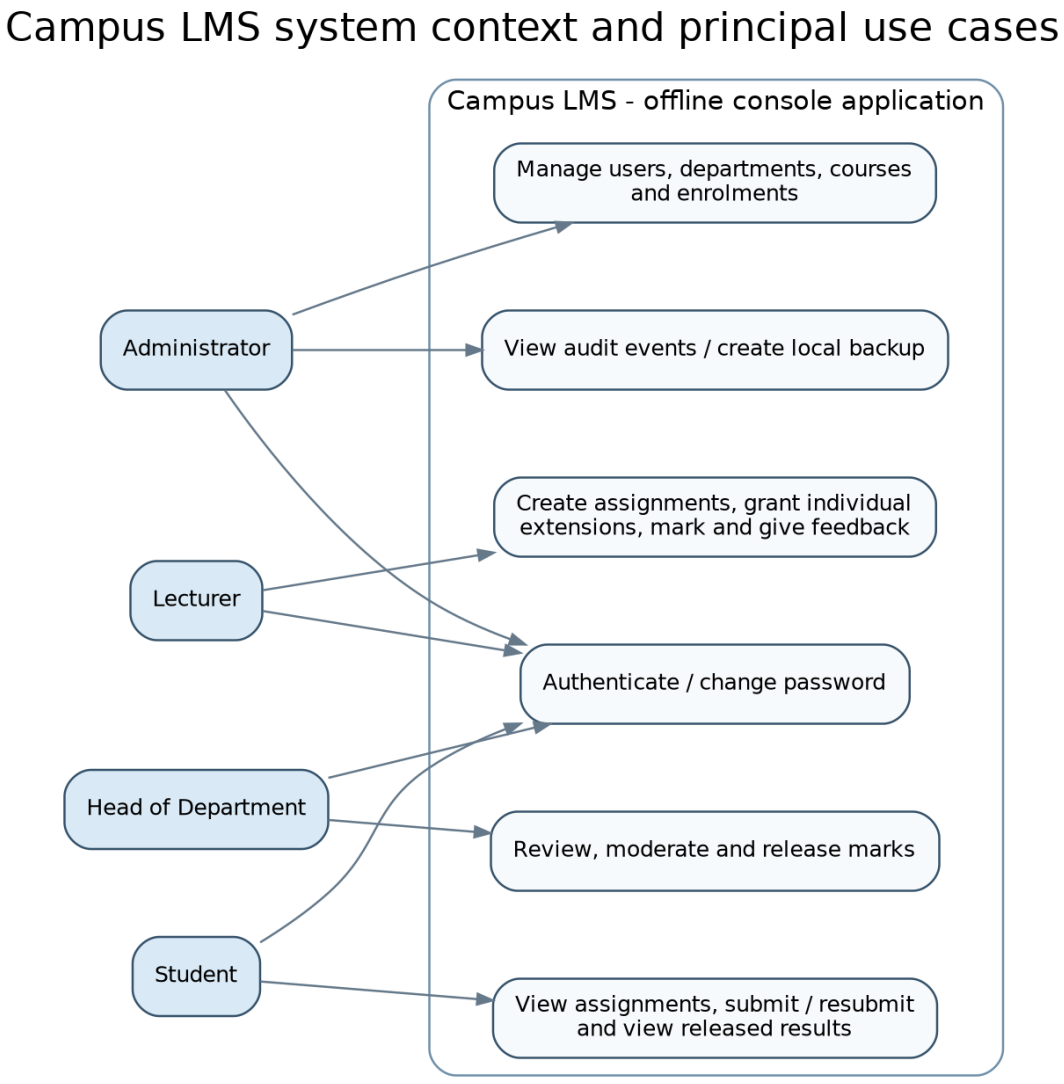


Figure 1. System context and main use cases for the four operational roles.

UC-01 Submit or resubmit assignment

Primary actor: Student.

Supporting components: LmsApplication, Database/SQLite and local file system.

Preconditions: Student is authenticated and active; the student is enrolled in the course; the assignment is available to that student; the application data and uploads folders are writable.

Trigger: The Student selects the submit/resubmit option from the Student menu.

Main success flow: 1. The application lists assignments in the Student's enrolled courses and displays the effective deadline. 2. The Student selects an assignment identifier. 3. The application confirms that the selected assignment is in the Student's available list. 4. The application checks whether an existing submission exists. 5. If no submission exists, the Student enters a full file path. 6. The application verifies that the path exists and is a regular file. 7. The application copies the file to data/uploads using a generated destination name. 8. The application calculates whether the submission is on time or late. 9. The application inserts the submission record with a bound SQL statement. 10. The application writes an audit event and reports the saved status and stored path.

Alternative flow A1 - resubmission: If an existing row is Submitted or Late, the application informs the Student that the stored file will be replaced and asks for confirmation. On confirmation it copies the new file and updates the current row, clears the previous mark, moderated mark and feedback, and recalculates status. If the Student declines, no database change is made.

Exception flow E1 - assignment unavailable: If the selected assignment is not in the Student's assignment list, the application stops the operation and reports that the assignment is not available to the account.

Exception flow E2 - invalid file: If the entered path does not exist or is a directory rather than a regular file, the application reports an error and does not create a submission record.

Exception flow E3 - already marked: If the existing submission is Marked, Moderated or Released, resubmission is blocked.

Postconditions: On success, a local copy of the document exists in data/uploads and one current submission row references it with status Submitted or Late. An audit event records the submission action. On cancellation or failure, no new submission row is committed; a copied destination file is removed if the later database write fails.

Business rules: BR-01 Only enrolled Students may submit. BR-02 The source must be a regular file. BR-03 A resubmission is allowed only while the existing row is Submitted or Late. BR-04 A submitted file is copied, not moved. BR-05 Late classification uses the effective individual deadline. BR-06 The current implementation replaces the prior submission reference and does not retain a version-history row.

UC-02 Lecturer marks a submission

Primary actor: Lecturer.

Preconditions: Lecturer is authenticated and active; the course is assigned to the lecturer; at least one submission is in Submitted or Late status.

Trigger: Lecturer chooses the marking option.

Main success flow: 1. The application queries submissions for courses assigned to the lecturer. 2. It lists eligible submissions with student, course, assignment, file, deadline and status information. 3. Lecturer selects a submission from that list. 4. The application validates that the selected item belongs to the lecturer's assigned courses. 5. Lecturer enters a mark between zero and the assignment maximum. 6. Lecturer enters written feedback. 7. The application calculates counted late days from the submission timestamp and effective deadline. 8. The system applies the penalty: 10% of the assignment maximum per counted day up to three days,

and zero beyond three counted days. 9. Lecturer confirms the result. 10. The application updates the mark, feedback and status to Marked, and writes an audit event describing the entered mark, late days, deduction and final mark.

Alternative flow A1 - cancellation: If the lecturer does not confirm the final mark, the update is not performed.

Exception flow E1 - unauthorised course: If the selected submission is not in the lecturer's assigned courses, the application rejects the operation.

Exception flow E2 - invalid mark: A negative value or a value greater than the maximum is rejected by the input range validation.

Exception flow E3 - status changed: If the database update affects no eligible row because the submission status changed, the application reports that the submission could not be marked.

Postconditions: On success, mark and feedback are stored, status becomes Marked and the item is available for HoD moderation. The student results query does not expose it yet.

Business rules: BR-07 Only assigned lecturers can mark course submissions. BR-08 Mark is constrained to zero through maximum mark. BR-09 Late penalty is based on effective deadline. BR-10 Lecturer marking does not release results to students.

UC-03 HoD moderates and releases a mark

Primary actor: Head of Department.

Preconditions: HoD is authenticated and active; the account has an assigned department; a submission in that department is Marked and contains a lecturer mark.

Trigger: HoD opens the moderation queue.

Main success flow: 1. The application retrieves Marked submissions for the HoD's assigned department. 2. HoD selects a submission. 3. The application confirms that the selected record belongs to the moderation queue and has a lecturer mark. 4. HoD enters a moderated mark between zero and the assignment maximum. 5. HoD records a moderation reason. 6. The application writes the moderated mark and changes status to Moderated. 7. The moderated queue displays the item. 8. HoD selects the item for release and confirms the action. 9. The application changes status to Released and writes a release audit event. 10. The Student may now view the released mark and feedback.

Alternative flow A1 - moderation review only: HoD may leave the item in the Moderated queue without releasing it. Student visibility remains blocked.

Exception flow E1 - outside department: A submission not returned by the HoD department query is rejected.

Exception flow E2 - missing mark: A submission without a lecturer mark cannot be moderated.

Exception flow E3 - invalid moderated mark: A value below zero or above the maximum is rejected.

Postconditions: After moderation, the submission has a moderated mark and Moderated status. After release, the state is Released and the Student results query may return it.

Business rules: BR-11 HoD moderation is department-scoped. BR-12 A moderated mark and a reason are required. BR-13 Only a Moderated record may be released. BR-14 The Student view returns Released records only.

4. Software architecture and detailed design

The observed release is a single-process console application. LmsApplication coordinates authentication, role menus, administration, assessment and file handling. Database owns the SQLite connection, schema and backup

behaviour; Statement wraps prepared-statement operations; PasswordHasher handles credentials; and small structures carry user, assignment and submission data. Figure 2 shows the actual, not an idealised, structure.

Some responsibilities are encapsulated: Database manages connection lifetime, Statement finalises SQLite statements and PasswordHasher isolates credential derivation. However, many SQL operations and business workflows remain inside LmsApplication. Presentation, validation, policy and persistence are coupled, which makes isolated tests and future extensions harder.

A layered architecture is a suitable future direction: console presentation, application services, domain rules and SQLite repositories. Services such as SubmissionService and ModerationService could depend on repository interfaces. The current source does not contain those layers or interfaces; this is a refactoring recommendation, not a description of the existing release.

A Qt GUI or web service was not selected. The assignment permits a console program and excludes web/mobile interfaces from scope. The console choice reduces runtime dependencies and supports offline operation. Its costs are limited discoverability, weaker accessibility and less separation between workflows and data access. The design trade-off is acceptable for a local prototype but not ideal for long-term expansion.

The inspected types are PasswordHasher, Statement, Database, User, AssignmentRow, SubmissionRow and LmsApplication. User is a structure containing an ID, username, role string and department ID. Role-specific menus are selected through role checks. This is not an abstract User base class with derived Administrator, Lecturer, HoD and Student classes, and it does not demonstrate polymorphic dispatch.

The assignment expects a design-level class diagram with visibility, types, multiplicities, inheritance and interfaces. The current source only supports a smaller diagram of helper classes and records. Additional headers were later mentioned by the student, but their contents and their integration into the CMake target were not available for this inspection. The source must be rebuilt after any such refactor.

The Statement wrapper demonstrates RAII: it is non-copyable, movable and finalises the SQLite statement in its destructor. Database also closes its connection in its destructor. Queries use bound parameters. These are sound implementation techniques, but the inspected source does not implement a separate Repository pattern or a Strategy interface for late penalties; the penalty calculation is a function and marking-workflow logic.

SOLID review identifies limitations. LmsApplication has multiple reasons to change; adding roles requires edits to role-string and menu logic; and there are no service/repository interfaces that demonstrate dependency inversion. The small helper classes have clearer responsibilities. A future refactor should isolate services and persistence and introduce polymorphic role behaviour where it adds value.

The database has eight main tables: departments, users, courses, enrollments, assignments, assignment_extensions, submissions and audit_logs. Enrollments model the student/course relationship; extensions record one assignment/student deadline and reason; submissions store the current submitted file, marks, feedback and status; audit_logs record the actor and event. Figure 3 summarises the foreign-key relationships.

Primary keys, foreign keys, uniqueness and check constraints protect the main relationships. A student/course pair and assignment/student pair are unique; roles and active flags are constrained; maximum marks must be positive and marks cannot be negative. Indexes support lecturer, student, assignment and submission lookups. A formal 3NF proof and full data dictionary were not supplied.

Submission status moves from Submitted or Late to Marked, then Moderated, then Released. The lecturer enters a mark and feedback; the HoD sets a moderated mark and reason; the HoD explicitly releases it. The student-results query filters for Released. Resubmission loops within Submitted/Late only, replacing the stored file reference. Figure 4 represents the observed state transitions.

A resubmission does not create a separate version-history row. The current record is updated and its mark, moderated mark and feedback are cleared. This permits correction before marking but does not preserve every

earlier version for later investigation. A future submission_versions table should record version number, file path, timestamp and uploader.

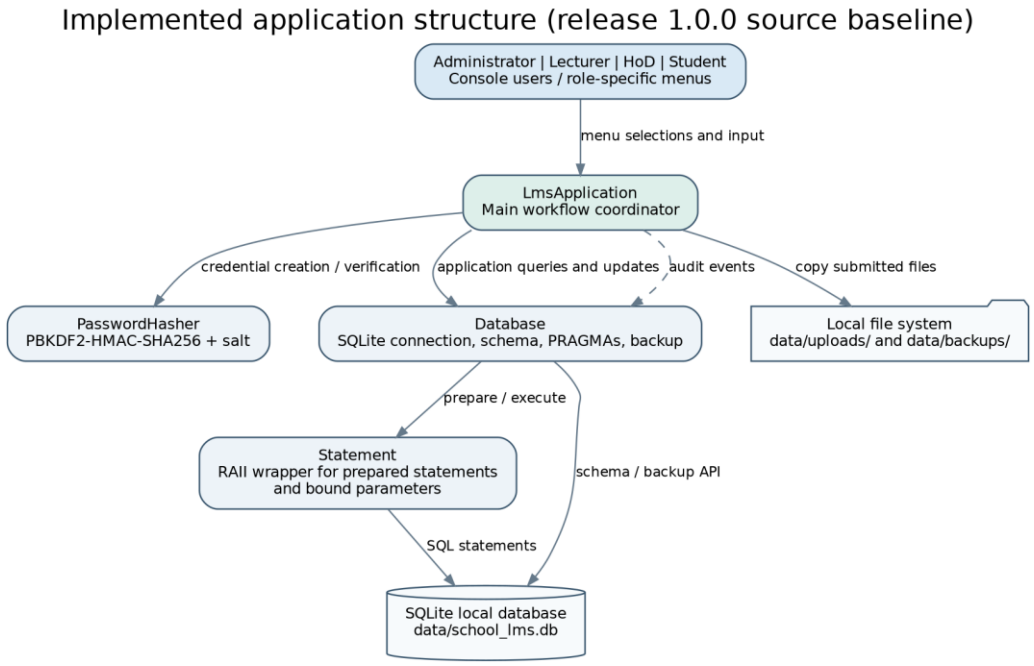


Figure 2. Implemented software structure in the inspected release 1.0.0 source baseline.

4.1 Observed relational data model

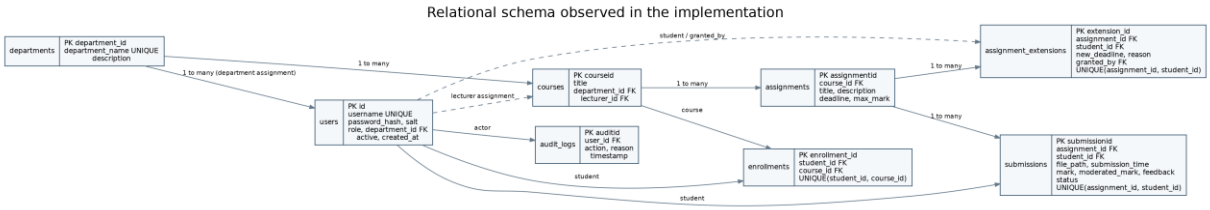


Figure 3. Entity-relationship overview reconstructed from the CREATE TABLE statements.

4.2 Submission state model

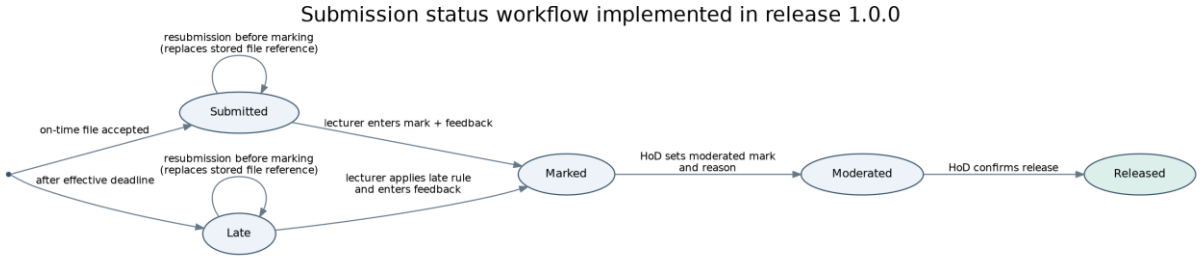


Figure 4. Status transitions represented by the current submission workflow.

4.3 Submission sequence overview

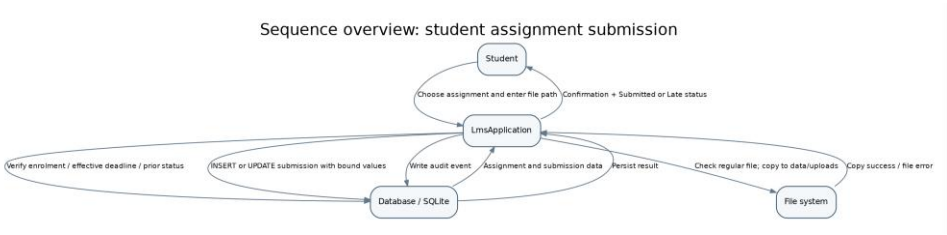


Figure 5. Main interactions when a student submits a local file.

5. Implementation, database and security

The project uses C++17 and CMake. CMake disables compiler extensions, builds the bundled SQLite amalgamation when available, and links Bcrypt on Windows or OpenSSL Crypto on other platforms. The executable target is CampusLMS and its inspected source is src/main.cpp. Warning flags are enabled for MSVC and non-MSVC compilers. The CMake CXX_STANDARD property supports C++17 [2].

The Windows configuration used CMake 4.5.0-rc1, MSVC 19.51, Visual Studio 18 2026 and Windows SDK 10.0.26100.0. CMake configuration completed, and the Release build generated sqlite3_local.lib and build/Release/CampusLMS.exe. This confirms the build on the development workstation, not portability on a separate clean PC.

Database opens data/school_lms.db relative to the process working directory, enables foreign keys, uses WAL mode and NORMAL synchronous mode, and sets a five-second busy timeout. The schema is created at startup and additive migrations add selected columns for earlier databases. SQLite prepared statements are compiled, bound, stepped and finalised through the C interface [3].

Bound parameters keep user values separate from SQL syntax, reducing injection risk and quoting errors. The schema enables foreign-key enforcement on the connection [4]. Database backup uses the SQLite backup API, and the application also copies the uploads folder. If that file copy fails, the database backup is removed so that an incomplete pair is not left behind.

Passwords are stored as a PBKDF2-HMAC-SHA256 derived value, not plain text. The implementation generates a 16-byte random salt and derives a 32-byte hash using 600,000 iterations. On Windows, BCryptGenRandom and BCryptDeriveKeyPBKDF2 provide the random salt and key derivation [5], [6]. The database stores salt and hash separately; password verification derives and compares the result.

On an empty users table, first-run setup creates a Computing department, demo accounts for the four roles, an introductory course, a student enrolment and a sample assignment. Demonstration credentials are for evaluation only and must be changed before real use. A school deployment should create individual accounts and avoid shared credentials.

Lecturers can create assignments only for courses assigned to them. The form accepts a title, optional description, deadline and positive maximum mark. Individual extensions require an enrolled active student, a later deadline and a reason; the Student assignment list displays the effective deadline for that student.

A Student may submit only an available assignment in an enrolled course. The source path must identify an existing regular file. The program copies the file into data/uploads with a generated name, then inserts a submission record. Resubmission replaces the current file reference and clears prior marks/feedback. Data paths are relative to the process working directory.

Lateness is calculated as the ceiling of elapsed seconds divided by 86,400. The lecturer enters a mark before penalty; the program deducts 10% of the assignment maximum per counted day for up to three days, and assigns

zero when lateness exceeds three counted days. The mark and feedback are saved as Marked. HoD moderation and release then control student visibility.

audit_logs stores the actor user ID, action, reason and timestamp. Significant operations write events, and marking details are included in the reason text. However, old and new marks are not separate structured fields. This only partially satisfies the requirement to capture who changed a mark, when, the old value, new value and reason. A dedicated mark-history table is recommended.

The runtime data directory contains the database, submitted files and backups. It should be excluded from source control. Public repositories must not contain live learner records, submitted assignments, passwords or database backups. Operating-system permissions should protect both the live data directory and backup copies.

6. Verification and validation

The V-Model links requirements to acceptance/system tests, design to integration tests and detailed functions to unit tests. For this application, tests should cover role boundaries and the assessment state transitions as well as successful use. The most important negative cases concern an unassigned lecturer, an HoD outside the department, un-enrolled students, invalid files and student access before release.

The test strategy should combine requirements-based tests, equivalence partitioning, boundary-value analysis and state-transition tests. Input partitions include valid/invalid passwords, menu choices, marks, dates and file paths. State-transition tests should cover first submission, permitted resubmission, marking, moderation and release, including attempts to move backwards or skip a state.

Deadline testing should include one second before, exactly at and one second after the deadline, plus exact and just-over 24-, 48- and 72-hour boundaries. The ceiling calculation means even a one-second delay counts as one day. Mark tests should include zero, maximum, below zero and above maximum. These cases are proposed; measured results were not supplied.

Entry criteria should require a successful build, deterministic test data and a disposable database. Integration tests should exercise schema constraints and persistence; system tests should use all role accounts; acceptance tests should follow the end-to-end assessment workflow. Exit criteria should require all mandatory cases executed, no unresolved critical/high defects, database integrity checked and residual limitations documented. The current project lacks an automated test target, so this is a proposed plan.

Observed manual evidence includes successful CMake configuration and Release compilation; application startup; Administrator login; Lecturer account creation and listing; Inno Setup compilation; matching SHA-256 values for source and installed databases; installed Administrator login; and account creation in the installed application. Student file submission succeeded after the entered path was supplied without surrounding quotation marks. These checks do not constitute an automated test suite.

A path usability issue was exposed: the program expects a file path, not a folder, and surrounding quotation marks were treated as part of the path. The user entered the same valid path without quotes and the submission succeeded. A low-severity improvement is to strip one matching pair of outer quotes before path validation and add tests for spaces, quotes and inaccessible files.

Test status is reported conservatively. Pass means a result was directly observed; Not run means no execution result was supplied; Improvement required means the test exposed a usability or compliance gap. Source inspection alone is not enough to mark a test Passed. Deadline boundaries, resubmission gates, moderation permissions, restore behaviour and clean-machine portability still need recorded execution.

No GoogleTest or Catch2 target, CI workflow or coverage report was found in the inspected source package. Therefore no automated test count or line-coverage percentage is claimed. Priority test targets include date parsing, lateDays boundaries, authentication, role checks, extensions, submission state transitions, database constraints and backup/restore. Coverage must be measured and reported using the actual run configuration.

System tests should exercise the full sequence: student submits, lecturer marks and gives feedback, HoD moderates and releases, and student views the released result. Negative tests should attempt cross-course lecturer access, cross-department HoD access and student viewing before release. Use a disposable database so that operational records are not changed.

The defect/action log records the quotation-mark path issue, incomplete old/new mark audit fields, missing automated test/CI evidence, absence of a polymorphic User hierarchy in the inspected source, open CR1 and the missing clean-machine install test. Each item needs an owner, correction, retest result and disposition. Process gaps are not runtime failures, but they affect assessment evidence.

Validation asks whether real users can complete their tasks, not only whether the code follows a branch. Acceptance sessions should observe whether staff can create assignments, mark work, moderate and release results, and whether students can locate and submit the right document. No formal user-acceptance session or accessibility evaluation was supplied, so those outcomes remain unverified.

6.1 Equivalence partitions and boundary values

Input / rule	Valid equivalence class	Invalid / boundary cases
Password	At least 8 characters.	Empty, 1–7 characters; verify stored value is not plain text.
Menu choice	Integer within the displayed menu range.	Text, decimal, negative, zero where disallowed, value above max.
Maximum mark	Finite numeric value greater than zero.	Zero, negative, non-number, NaN/Infinity, extremely high input.
Awarded mark	Finite value from 0 to assignment maximum.	Below 0; just above max; non-number; empty.
Deadline string	Valid calendar date/time in supported format.	Wrong format; impossible date; one second before/on/after deadline.
Lateness	0, 1, 2, 3 or more than 3 counted 24-hour periods.	1 second late; 24h exact; 24h+1s; 72h exact; 72h+1s.
Submission path	Existing regular file that can be copied.	Missing path, directory, inaccessible file, quoted path, copy collision.
Resubmission state	Existing status Submitted or Late.	Marked, Moderated or Released; cancellation at confirmation prompt.

6.2 V-Model test levels

V-model level	Artefact / behaviour verified	Entry criteria	Exit criteria
Unit	Date parser, lateDays, password helper, input validation and status rules.	Deterministic inputs, isolated functions and test fixtures.	All planned unit cases pass; boundary results recorded.
Integration	Database schema, prepared statements, extensions, submissions, audit and backup.	Temporary SQLite DB and controlled test data.	Relationships and transactions verified; no unresolved high defects.
System	Complete role workflows and executable on target Windows environment.	Release build, seeded or disposable data, role accounts.	Mandatory system scenarios pass and errors are documented.
Acceptance	School user tasks: enrolment, submission, marking, moderation and release.	Agreed workflow, role accounts and user representatives.	Users complete agreed tasks; residual risks and usability issues accepted.

6.3 Test execution record

ID	Test	Expected result	Observed status
T-01	CMake configure on Windows	CMake generates Visual Studio build files and completes configuration.	Pass - output showed configure/generate completed.
T-02	Release compilation	SQLite static library and CampusLMS.exe are produced without errors.	Pass - sqlite3_local.lib and CampusLMS.exe generated.
T-03	First launch	Main menu appears and local database/demo setup is created when empty.	Pass - user observed startup message and menu.

ID	Test	Expected result	Observed status
T-04	Administrator authentication	Valid admin account opens Administrator Dashboard.	Pass - observed on development and installed application.
T-05	Create Lecturer account	Valid username and password create a Lecturer account.	Pass - account was created and listed with an ID.
T-06	Installed database copy	Installed DB bytes match source DB at time of copy.	Pass - SHA-256 hashes matched.
T-07	Installed write path	Installed application can create a user and display it.	Pass - user reported successful creation and list display.
T-08	Submit a regular file	Existing PDF file is copied and submission is saved.	Pass - user reported success after removing surrounding quotation marks.
T-09	Reject directory as file	A folder path is rejected as not a regular file.	Pass - application reported expected validation error.
T-10	Quoted path robustness	A copied path surrounded by quotes is accepted or clearly normalised.	Improvement required - quotes caused path-not-found; unquoted path worked.
T-11	Late penalty boundaries	Test before/on/after deadline and day 1/day 2/day 3/day 4 boundaries.	Not run - automated or recorded manual boundary results not supplied.
T-12	Resubmission gate	Submitted/Late may be replaced; Marked/Moderated/Released may not.	Not run - source logic inspected; test result not supplied.
T-13	HoD moderation/release	Only department Marked rows can be moderated and Released.	Not run - source logic inspected; end-to-end result not supplied.
T-14	Restore backup set	Database and uploads restore to a consistent usable state.	Not run - backup operation exists; restore evidence not supplied.
T-15	Clean-machine installation	Installer launches on a second clean Windows PC without developer tools.	Not run - only same-machine installation was checked.
T-16	Automated unit/integration suite	GoogleTest or Catch2 suite runs through CTest and CI.	Not run - no test target or CI workflow in inspected project package.

6.4 Defect and action log

ID	Severity	Finding / cause	Action / retest
D-01	Low	Copied file paths containing surrounding double quotation marks were interpreted with the quotes as part of the path.	Workaround confirmed: enter path without surrounding quotes. Recommended fix: trim matching outer quote pair, add tests for path with spaces and quoted path.
D-02	High / compliance	The inspected audit schema records actor, action, reason and time but no dedicated old/new mark fields.	Add immutable mark-history records or old_mark/new_mark columns; test lecturer mark and HoD moderation changes.
D-03	High / evidence	No GoogleTest/Catch2 target, CI workflow or coverage report was present in the inspected source package.	Add test target and CI; execute suite; publish coverage measurement and run logs.
D-04	High / design	The inspected code uses a User structure and role-specific menu checks, not an abstract User hierarchy with polymorphism.	Refactor only after role headers are reviewed; add derived roles and virtual behaviour; rebuild and rerun regressions.
D-05	Medium / maintenance	CR1 class-wide extensions is not implemented in release 1.0.0.	Complete impact analysis, transactional update and student-list display; test precedence and rollback; release as v1.1 only after retest.
D-06	Medium / deployment	Only same-machine installation was verified.	Install on a clean Windows desktop/account and verify runtime, write access, restart persistence and backup/restore.

7. Configuration management and quality assurance

The project has CMake configuration, Windows build scripts, a README, SQLite source and an Inno Setup script. The student supplied the repository URL https://github.com/LibenyanePosholi/CampusLMS_CPP and reported uploading the project files through GitHub. The URL should be included in the submission; the exact commit identifier and anonymous public access should be checked before hand-in.

A suitable workflow is a protected main branch with short-lived feature and bug-fix branches, descriptive commit messages and tags for tested releases. Git for Windows 2.56.0.4 was installed and verified from PowerShell; the

student used GitHub's web interface to upload files. No local commit history or release tag was supplied for inspection. A v1.1 tag should wait until CR1 is implemented and regression-tested.

The assignment requires CI. A GitHub Actions workflow should configure CMake, build the project and run tests on each push and pull request. Windows should be included because it is the target platform. No workflow file or CI execution evidence was present in the inspected package, so CI remains outstanding.

The CMake file enables compiler warnings, but no dedicated static-analysis report was supplied. cppcheck or clang-tidy should be run with a documented configuration; findings should be fixed, justified or deferred. Useful metrics include lines of code, function count, cyclomatic complexity and duplicated logic. The large LmsApplication class should be a priority for review and refactoring.

A code review should check input validation, parameter binding, resource lifetime, role scoping, deadline calculations, filesystem behaviour, backups and audit completeness. The source uses RAII for SQLite statements and validates numeric/date inputs. Further review should focus on the monolithic workflow class, old/new mark auditing, submission version history and repeatable tests.

8. Deployment and operational documentation

Deployment targets one Windows desktop. The Inno Setup 7.1.1 compiler generated installer-output/CampusLMS_Setup.exe from CampusLMS_Setup_Updated.iss. The recorded output size was 2,644,945 bytes. The installed folder contains CampusLMS.exe, the local database, uploads directory and uninstaller. Shortcuts specify the application folder as the working directory.

The installed application was found under the user's Downloads\CampusLMS folder. Its database was 90,112 bytes, and SHA-256 comparison with the development database produced matching hashes. The installed program launched, authenticated the Administrator and accepted a new Lecturer account. This confirms same-machine installation and basic write operation, not clean-machine portability.

The installer was tested on the development computer only. A separate clean Windows desktop or account should be used to confirm that no development tools are required, that the shortcut sets the working directory, that files can be written, and that records persist after closing and relaunching. The database and uploads must be backed up before additional installer tests.

The technician should run the setup file, confirm the destination, complete installation and launch using the created shortcut. The technician should verify that data/school_lms.db and data/uploads exist and are writable, replace demonstration credentials and create individual accounts. The application should not be launched with an unrelated working directory because it resolves data paths relative to the current directory.

The role manuals should mirror the real menus. Administrator instructions cover accounts, departments, courses, lecturer/HoD assignment, enrolment, audit review, backup and password changes. Lecturer instructions cover assigned courses, assignment creation, marking, feedback and individual extensions. HoD instructions cover departmental moderation and release. Student instructions cover registration, assignments, local file submission/resubmission and released results.

The Administrator backup action creates a database backup and a matching uploads copy. The school should store both on protected external media, restore them periodically into a test folder and confirm file references. A database-only copy is incomplete if submitted files are missing. Retention and deletion rules should be agreed by the institution.

Release 1.0.0 should describe role menus, offline SQLite persistence, assignment workflow, individual extensions, marking, moderation, release, audit events and local backup. Known limits include the console interface, single-computer operation, lack of class-wide extensions, no retained submission versions, incomplete structured mark auditing and missing automated test/CI evidence. Figure 6 shows the actual deployment topology.

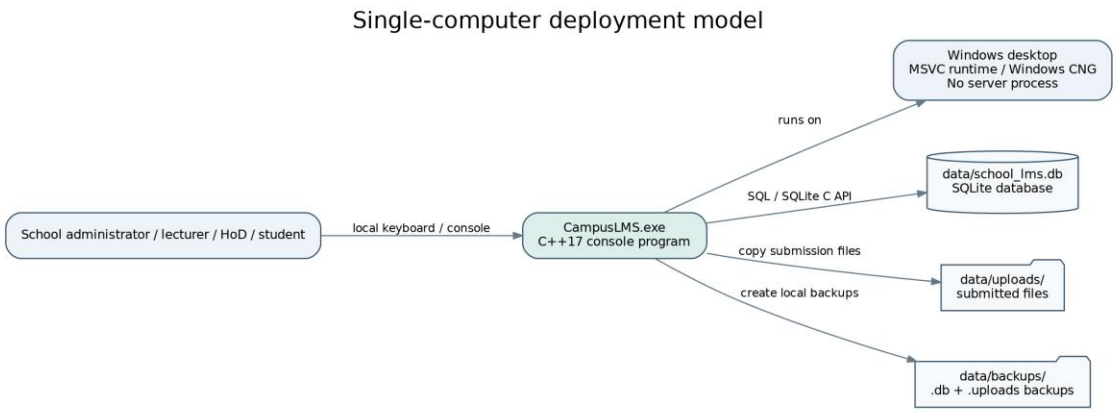


Figure 6. Deployment topology for the one-computer offline installation.

8.1 Installation and first-run checklist

1. Confirm that the target Windows account is authorised to install and run the application.
2. Run CampusLMS_Setup.exe and review the selected installation folder before accepting it.
3. Complete installation and launch through the created shortcut so that the working directory is the application directory.
4. Confirm that data/school_lms.db and data/uploads exist and are writable.
5. Sign in with the demonstration Administrator account only for evaluation, then change its password.
6. Create individual staff and student accounts with minimum necessary roles.
7. Create a backup set and perform a restore test in a separate directory before live use.

9. Software maintenance and change request CR1

CR1 asks lecturers to extend an assignment deadline for the whole class and display the revised deadline in student assignment lists. This is perfective maintenance because it adds to an existing individual-extension capability. It changes the lecturer workflow, deadline persistence and student display and requires regression testing.

Release 1.0.0 supports individual extensions: the lecturer selects an enrolled student and assignment, enters a later deadline and reason, and the student list displays that effective deadline. CR1 must define precedence where an individual extension already exists and whether future enrolments inherit the class extension. These policy rules should be agreed before implementation.

CR1 affects the Lecturer use case and menu, extension business logic, assignment_extensions rows, student assignment query, audit logging, transaction handling and tests. A class-wide operation should validate lecturer ownership and all affected students, then apply updates in one transaction so a mid-operation failure cannot leave only some students extended. The audit record should include actor, assignment, reason, deadline change and affected count.

CR1 tests should cover all enrolled students receiving the deadline, lecturer ownership, a deadline later than the current effective deadline, existing individual extensions, a course with no students, transaction rollback, student-list display and regression of individual extensions. The full existing suite must be rerun before release.

CR1 is not present in the inspected release 1.0.0 source, and no v1.1 tag, regression output or release record was supplied. It remains open. A v1.1 label should only be used after code, design documents, tests, release notes and repository tag have been updated.

9.1 CR1 change impact matrix

Affected item	Required change for CR1
Requirement / use case	Add lecturer action to extend an assignment for all enrolled students; define precedence with existing individual extensions.
Menu / presentation	Add class-wide extension choice, summary of affected students and confirmation before applying.
Business logic	Validate lecturer ownership, new deadline, enrolment set and policy for existing individual exceptions.
Database	Use assignment_extensions rows within a transaction; add consistent reason and audit detail.
Student assignment list	Continue to display effective deadline for each student after class-wide update.
Testing	Add positive, negative, boundary, zero-enrolment, existing-extension and rollback tests; rerun full regression suite.

10. Professional, legal and ethical considerations

Campus LMS stores usernames, role assignments, departments, enrolments, submissions and marks. Submitted documents may contain additional personal information. Lesotho's Data Protection Act, 2011 (Act No. 5 of 2012) applies to relevant data controllers and addresses lawful processing, minimality, purpose, retention, security and compromise notification [7]. This report is not legal advice or a compliance certificate.

Role-based access protects confidentiality and fairness. Students should see their own enrolments and released results; lecturers should work within assigned courses; HoDs should moderate within their department; and administrators manage accounts and structure. The source applies role checks and scoped queries, but a negative authorisation test suite is still needed.

The salted PBKDF2 design avoids plain-text password storage, but operational security also requires changed demo credentials, separate accounts, Windows file permissions and protected backups. A user who can copy the SQLite file may inspect or alter data outside the application. Physical security and operating-system controls remain necessary.

The penalty counts a positive delay as a started 24-hour period. A submission seconds late therefore receives one counted day. This may be severe unless the school approves and communicates the interpretation. Extension reasons should be recorded, and the school should define how exceptional circumstances and appeals are handled.

Moderation improves consistency only if decisions remain explainable. Current audit entries record actor, action, reason and time, but old and new marks are not structured fields. A dedicated mark-history table should preserve each change without overwriting earlier decisions. Students should receive written feedback after release.

The console supports keyboard input, but accessibility features such as screen-reader labelling, scalable text, colour contrast evaluation and alternative interaction were not assessed. A future interface should be evaluated with users with disabilities and staff who are not confident with computers.

Third-party SQLite source and any future libraries must retain applicable licence notices. Public source control must exclude live records, credentials, uploads and backups. Academic reporting must distinguish manual outcomes from automated test evidence and must not reuse test statistics, diagrams or claims from another project.

11. Critical evaluation and conclusion

Campus LMS demonstrates an offline role-based workflow, persistent SQLite storage, parameterised statements, salted password derivation, individual extensions, submission copying, late marking, HoD moderation/release and

local backup. The Windows Release build and installer succeeded; the installed database matched the source database by SHA-256; and the installed application accepted Administrator login and account creation.

The inspected source remains monolithic and lacks the assignment's abstract polymorphic User hierarchy and two explicit design patterns. Audit data is not fully structured for old/new marks, submission versions are not retained, and automated tests, CI, coverage and static-analysis evidence are absent. CR1 is not implemented. These are material compliance gaps, not cosmetic items.

The appropriate status is a working release 1.0.0 prototype with successful same-machine Windows build and installation, not a fully evidenced production system. Priorities are to inspect and integrate role headers, refactor the user and service model, strengthen audit history, add tests and CI, complete static analysis, implement CR1 as v1.1 and finish manuals and traceability.

Campus LMS is a credible foundation for the moderated assessment case and demonstrates practical C++17, SQLite, CMake and Windows packaging. Its value is best presented with transparent evidence: implemented features, observed manual outcomes and unresolved limitations. This provides a reviewable baseline and a clear improvement path toward maintainability, testability and accountability.

Appendix A. Requirements traceability matrix

The matrix links major implemented capabilities to functional requirements and the implementation evidence inspected. Status values are deliberately cautious: "not supplied" means a test artefact was not available, not that the feature is necessarily broken.

Capability	Requirement IDs	Design / source / data elements	Verification evidence
User authentication / role menus	FR-01, FR-02	PasswordHasher; LmsApplication login/register; users table	Manual admin login pass; other role tests not fully evidenced
User / department / course administration	FR-03 to FR-06	LmsApplication admin workflows; users/departments/courses/enrollments	Admin menu and account creation observed; complete workflow tests not supplied
Assignment creation / individual extension	FR-08, FR-09	createAssignment, grantExtension; assignments / assignment_extensions	Implementation inspected; dedicated test cases not supplied
Submission / resubmission	FR-10 to FR-13	submitAssignment; submissions; data/uploads	Manual file path behaviour observed; boundary tests not supplied
Marking / feedback / late penalty	FR-14	markSubmissions, lateDays; submissions	Implementation inspected; numerical boundary tests not supplied
Moderation / release	FR-15, FR-16	moderateMark, releaseMark, hodRows; submissions	Implementation inspected; end-to-end acceptance evidence incomplete
Audit events	FR-17	audit; audit_logs	Audit schema/action logging inspected; old/new dedicated mark fields absent
Backup	FR-19	backupData, Database::backupTo; data/backups	Implementation inspected; restore drill not supplied
CR1 class-wide extension	CR1	Would affect Lecturer workflow, extension logic, assignment list and transaction logic	Not implemented in inspected release 1.0.0 source

Appendix B. Verification evidence checklist

- Store the CMake configure output and Release build output with the date, compiler version and source commit identifier.
- Store the Inno Setup compiler output and the exact CampusLMS_Setup.exe file size or hash.
- Record the installed folder and the result of the database hash comparison. Do not include the live database in the public repository.
- Create a disposable database and record the end-to-end student submission → lecturer marking → HoD moderation → release → student view scenario.

- Create unit tests for deadline parsing, lateDays boundaries, mark bounds, input validation and PasswordHasher verification.
- Create database integration tests for constraints, user creation, course enrolment, individual extension and backup consistency.
- Add a GitHub Actions workflow that builds and runs the tests on each push or pull request.
- Generate the actual coverage report and report its measured numerator, denominator, filters and tool version.
- Run cppcheck or clang-tidy and retain the full output plus the disposition of each finding.
- After CR1 is implemented, update the source tag and release notes, add regression results and identify the exact v1.1 commit.

Appendix C. Build and installation guide

C.1 Build requirements

The documented Windows build environment requires Windows 10 or later, Visual Studio with the Desktop development with C++ workload, a Windows SDK, CMake 3.20 or later, and the SQLite amalgamation files in the SQLite 3 directory. In the development session, the installed versions were CMake 4.5.0-rc1, Microsoft Visual C++ 19.51 and Windows SDK 10.0.26100.0.

C.2 Configure and build

```
cmake -S . -B build
```

```
cmake --build build --config Release
```

The Release executable is generated at build/Release/CampusLMS.exe. The recommended start script changes the working directory to the project root before launching, so that the relative data/ directory is resolved correctly.

C.3 Compile the installer

With Inno Setup 7 installed in the current user profile, compile the updated installer script using:

```
& "$env:LOCALAPPDATA\Programs\Inno Setup 7\ISCC.exe" ".\CampusLMS_Setup_Updated.iss"
```

The observed output was installer-output/CampusLMS_Setup.exe. The installer includes the Release executable and the project database present when the script is compiled. The installer should be tested from a clean environment before school deployment.

Appendix D. Outstanding compliance actions before final submission

Priority	Action	Why it matters	Evidence to retain
1 - Critical	Inspect and integrate the recently created role header files; if the assignment requires OOP, implement an abstract User base class and derived role classes with virtual behaviour.	The brief explicitly requires abstract User, inheritance and polymorphism. Uploading unused headers alone does not satisfy this requirement.	Updated source, CMake build output and class diagram consistent with code.
2 - Critical	Add a structured mark-change history containing actor, timestamp, old mark, new mark and reason.	The assignment requires every mark change to be auditable in those terms.	Schema migration, audit service and tests.
3 - Critical	Add GoogleTest or Catch2 unit and integration tests, integrate with CTest and CI, and measure line coverage.	The Phase 5 automated testing item is worth 5 marks and requires actual coverage evidence.	Test code, CI run, logs and report.
4 - High	Implement CR1 class-wide extensions and publish a tested v1.1 release.	Phase 8 explicitly assesses the maintenance request and regression testing.	Change analysis, code, updated design, regression suite, tag/release.
5 - High	Complete formal architecture, class, behaviour and ERD diagrams and traceability against the final code.	The design and consistency criteria require the same system across artefacts.	Editable UML and traceability matrix.
6 - High	Complete a clean-machine installation test and	Same-machine installation is not	Installation checklist, logs, restore

Priority	Action	Why it matters	Evidence to retain
	restore test for database plus uploads.	independent portability or recovery verification.	check.
7 - Medium	Retain static-analysis output, code metrics, review checklist and documented dispositions.	Required for Phase 6 quality assurance.	cppcheck/clang-tidy output and metrics.
8 - Medium	Prepare and check role-specific user manuals and installation/release notes.	Required for Phase 7 deployment.	Administrator, Lecturer, HoD and Student manuals; release note.

References

[1] National University of Lesotho, Department of Mathematics and Computer Science, "CS4433 – Software Engineering – Individual Assignment," case study and project specification, 7 November 2026, supplied course handout.

[2] Kitware, "CXX_STANDARD — CMake Documentation," https://cmake.org/cmake/help/latest/prop_tgt/CXX_STANDARD.html (accessed 3 November 2026).

[3] SQLite, "An Introduction to the SQLite C/C++ Interface," https://www.sqlite.org/cintro.html (accessed 3 November 2026).

[4] SQLite, "PRAGMA Statements — foreign_keys," https://www.sqlite.org/pragma.html (accessed 3 November 2026).

[5] Microsoft, "BCryptDeriveKeyPBKDF2 function," Win32 API documentation, https://learn.microsoft.com/en-us/windows/win32/api/bcrypt/nf-bcrypt-bcryptderivekeypbkdf2 (accessed 3 November 2026).

[6] Microsoft, "BCryptGenRandom function," Win32 API documentation, https://learn.microsoft.com/en-us/windows/win32/api/bcrypt/nf-bcrypt-bcryptgenrandom (accessed 3 November 2026).

[7] Kingdom of Lesotho, Data Protection Act, 2011 (Act No. 5 of 2012), published in Government Gazette No. 19, 22 February 2012, https://lesotholii.org/akn/ls/act/2012/5/eng@2012-02-22 (accessed 3 November 2026).

[8] Campus LMS project, "README.md," project build, runtime, database and operational notes, release 1.0.0, project file supplied for this report.

[9] Campus LMS project, "src/main.cpp" and "CMakeLists.txt," source-code baseline inspected for this report, project archive supplied during the development conversation.

[10] LibenyanePosholi, "CampusLMS_CPP," GitHub repository, https://github.com/LibenyanePosholi/CampusLMS_CPP (repository address supplied by the student; verify commit and access status at submission).